AP CSP · UNIT 3 · CHEAT SHEET

Canvas math: placing things with loops

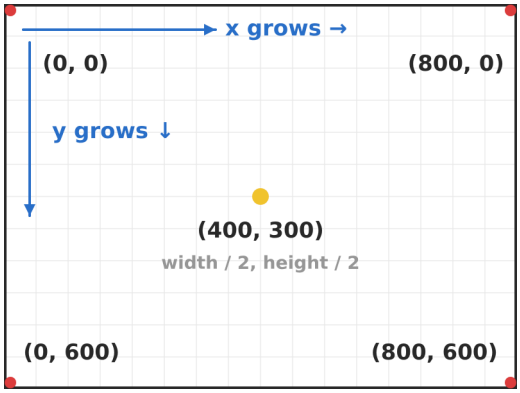
Every example uses createCanvas(800, 600). Use width and height instead of 800 and 600 so your code works on any canvas.

1 The canvas

`(0, 0)` is the **top-left** corner. x grows to the right. **y grows down**, not up.

Center of the canvas: `(width / 2, height / 2)` = `(400, 300)`.

Bottom-right corner: `(width, height)`. Anything with x > width or y > height is off screen.



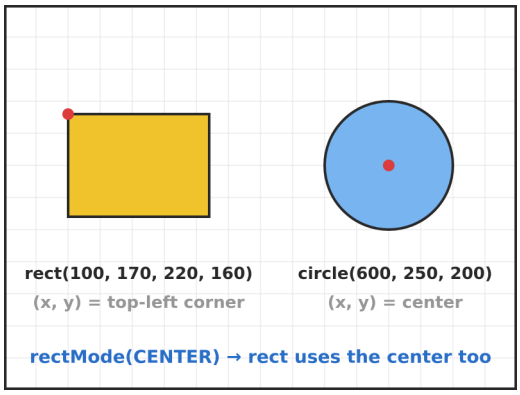
2 Corner or center?

`rect()`, `image()` and `text()` start at the **top-left corner**. `circle()` and `ellipse()` use the **center**.

To center a rect on (cx, cy) without rectMode:

```
rect(cx - w / 2, cy - h / 2, w, h);
```

Or switch modes once in setup: `rectMode(CENTER);`

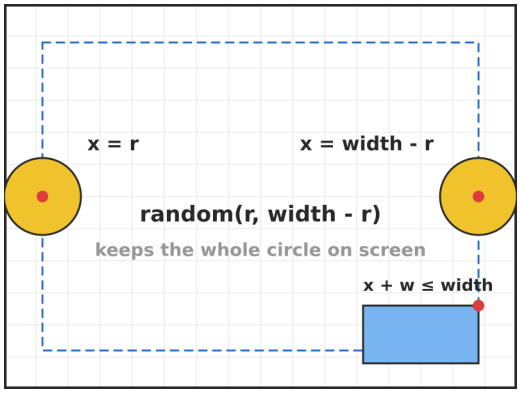


3 Stay on the screen

A circle with radius r touches the left edge when x = r, and the right edge when x = width - r.

```
let r = 60;
let x = random(r, width - r);
let y = random(r, height - r);
```

For a rect in corner mode: keep `x + w ≤ width` and `y + h ≤ height`.

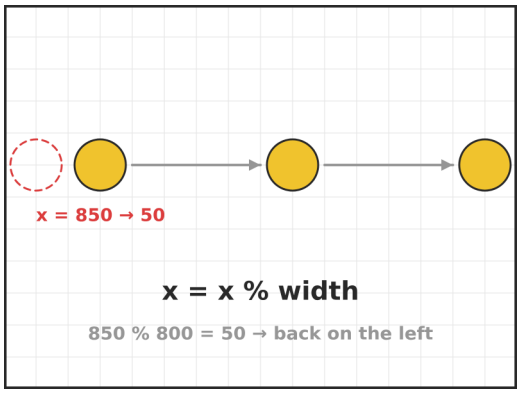


4 Wrap around the edge

`%` gives the remainder. Past the right edge? Come back on the left.

```
x = x + 3;
x = x % width;
```

To slide in and out instead of popping, wrap a wider range: **`(x + 50) % (width + 100) - 50`**. The asteroid rocks do this with y.

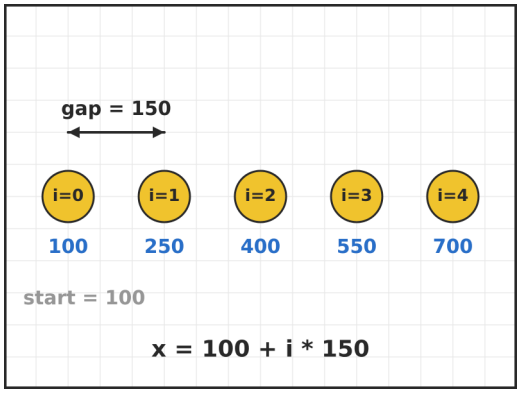


5 A row: start + i × gap

The loop variable **`i`** counts 0, 1, 2, … Multiply it by the gap to space things out.

```
for (let i = 0; i < 5; i++) {
  let x = 100 + i * 150;
  circle(x, 300, 80);
}
```

The **last** item is at **`start + (n - 1) * gap`**, not **`start + n * gap`**.

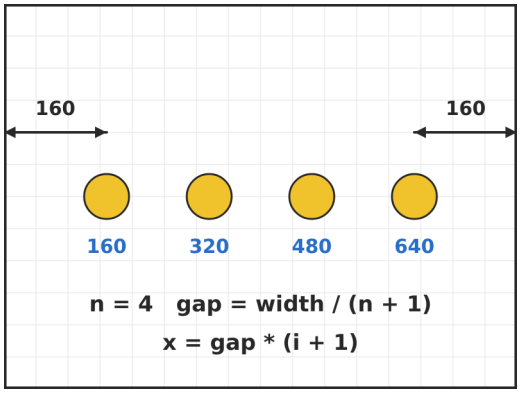


6 Spread n things evenly

Split the width into **n + 1** equal gaps. The items sit between them.

```
let n = 4;
let gap = width / (n + 1);
for (let i = 0; i < n; i++) {
  circle(gap * (i + 1), 300, 70);
}
```

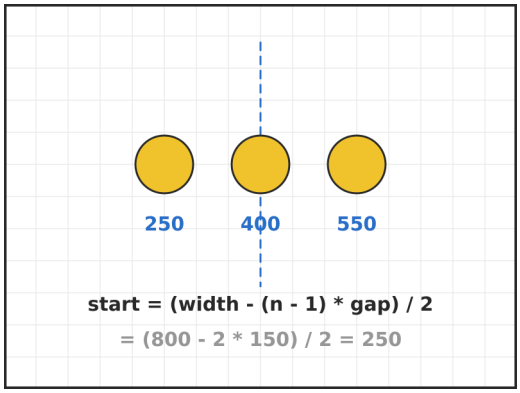
Change n and everything re-spaces itself.



7 Center a row

The row is **`(n - 1) * gap`** wide. Split the leftover space in half.

```
let n = 3;
let gap = 150;
let start = (width - (n - 1) * gap) / 2;
for (let i = 0; i < n; i++) {
  circle(start + i * gap, 250, 90);
}
```



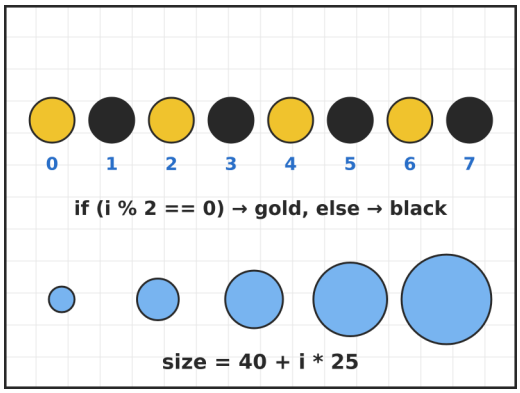
8 Patterns from i

Alternate: `i % 2` is 0, 1, 0, 1, …

```
if (i % 2 == 0) {
  fill(244, 196, 48);
} else {
  fill(43);
}
```

Grow: use i in the size: `circle(90 + i * 150, 460, 40 + i * 25);`

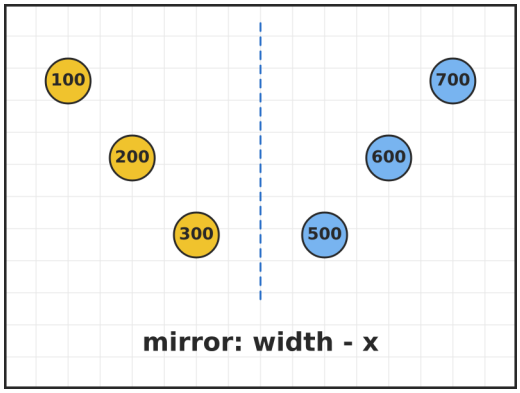
Every 3rd: `i % 3 == 0`. **Shade:** `fill(i * 30);`



9 Mirror

The mirror of x across the middle is `width - x`. Upside-down: `height - y`.

```
for (let i = 0; i < 3; i++) {
  let x = 100 + i * 100;
  let y = 120 + i * 120;
  circle(x, y, 70);
  circle(width - x, y, 70);
}
```

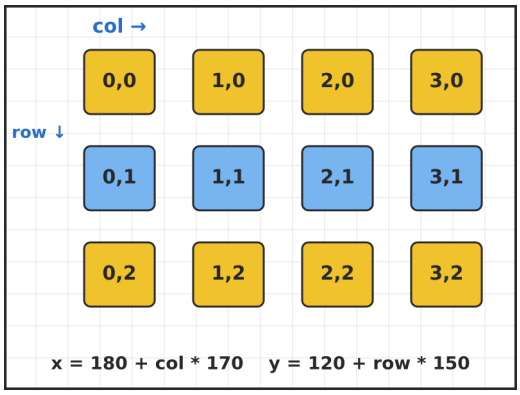


10 A grid: a loop inside a loop

The outer loop picks the row (y). The inner loop draws every column (x) in that row.

```
for (let row = 0; row < 3; row++) {
  for (let col = 0; col < 4; col++) {
    let x = 180 + col * 170;
    let y = 120 + row * 150;
    rect(x - 55, y - 50, 110, 100);
  }
}
```

Total drawn: rows × cols = 12.

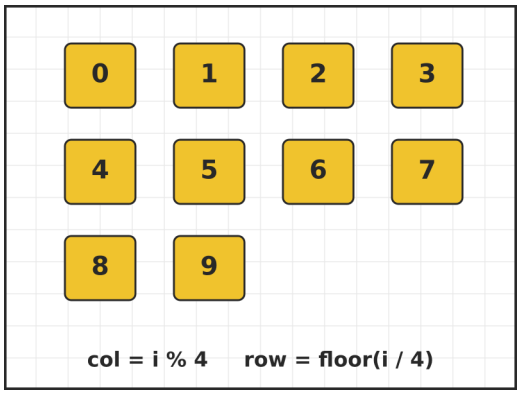


11 A grid from one loop

Number the items 0, 1, 2, … and work out the row and column from i.

```
for (let i = 0; i < 10; i++) {
  let col = i % 4;
  let row = floor(i / 4);
  let x = 95 + col * 170;
  let y = 60 + row * 150;
  rect(x, y, 110, 100);
}
```

Good when the count is not a full grid (10 items in rows of 4).

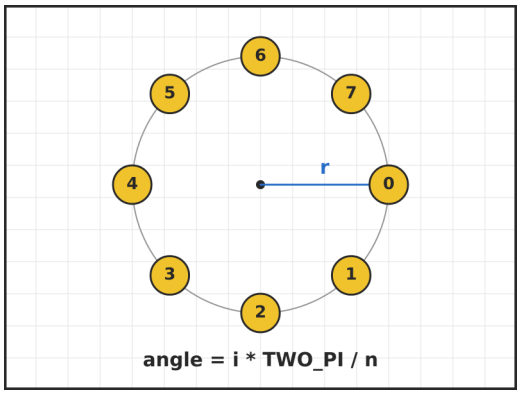


★ Going further: a ring

Spread n items around a circle of radius r centered at (cx, cy). Each one gets an equal slice of the full turn, **`TWO_PI`**.

```
let n = 8;
for (let i = 0; i < n; i++) {
  let angle = i * TWO_PI / n;
  let x = 400 + 200 * cos(angle);
  let y = 280 + 200 * sin(angle);
  circle(x, y, 60);
}
```

Angle 0 points right, and it turns **clockwise** because y grows down.



Quick reference

Goal	Formula
Center of canvas	`(width / 2, height / 2)`
Row with a gap	`x = start + i * gap`
Last item in a row	`start + (n - 1) * gap`
n items, even spacing	`gap = width / (n + 1);  x = gap * (i + 1)`
Center a row	`start = (width - (n - 1) * gap) / 2`
Center a rect (corner mode)	`rect(cx - w / 2, cy - h / 2, w, h)`
Random, fully on screen	`random(r, width - r)`
Wrap around	`x = x % width`
Every other one	`i % 2 == 0`
Mirror	`width - x`
Grid from one loop	`col = i % cols;  row = floor(i / cols)`
Ring of n	`angle = i * TWO_PI / n`

Common mistakes

- Using **800** and **600** instead of **`width`** and **`height`**.
- Forgetting that **`circle()`** is placed by its **center**, so half of it hangs off the edge at x = 0.
- Off by one: with **`i < n`**, i ends at n - 1. The last item is at **`start + (n - 1) * gap`**.
- A gap that is too big pushes the last items off screen. Check: last x ≤ width - r.
- Putting **`let x = 100;`** **inside** the loop when you meant to add to it each trip. Start values go before the loop.

AP CSP · UNIT 3 · CHEAT SHEET · PART 2

Compound shapes: push, pop and translate

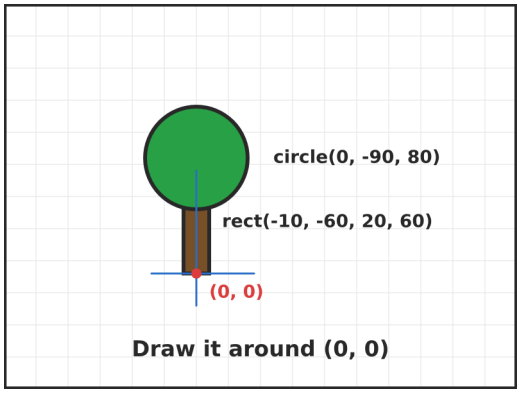
Draw a shape once around (0, 0), then stamp copies of it anywhere with a loop.

1 Draw it around (0, 0)

Design a compound shape in a function, with **(0, 0) at its base**. Negative y is up.

```
function drawTree() {
  fill(120, 80, 40);
  rect(-10, -60, 20, 60);    // trunk
  fill(40, 160, 70);
  circle(0, -90, 80);        // leaves
}
```

Call it with `drawTree();` and it draws at the top-left corner, until you move (0, 0).

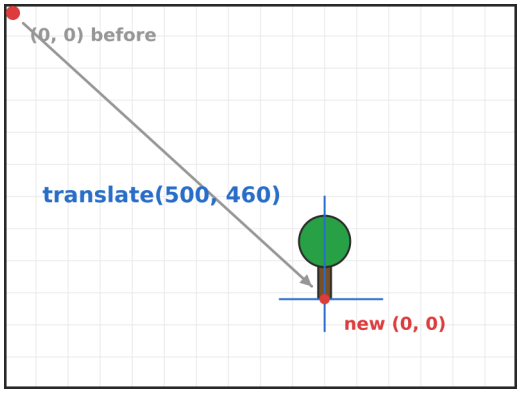


2 translate(x, y) moves (0, 0)

`translate()` moves the origin. Everything after it is drawn from the new (0, 0).

```
translate(500, 460);
drawTree();
```

You never change the numbers inside `drawTree()`. You move the paper, not the pencil.

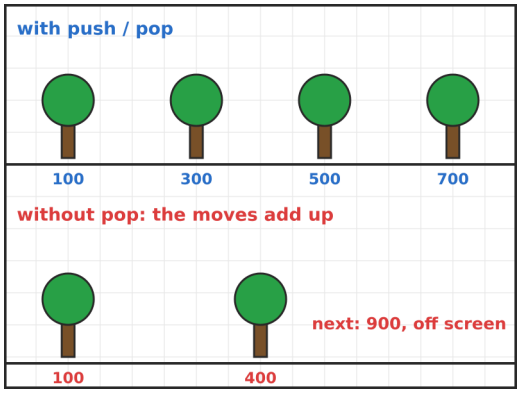


3 push() and pop()

translate adds up. Two `translate(200, 0)` calls move you 400. `push()` saves where you are; `pop()` goes back.

```
for (let i = 0; i < 4; i++) {
  push();
  translate(100 + i * 200, 240);
  drawTree();
  pop();
}
```

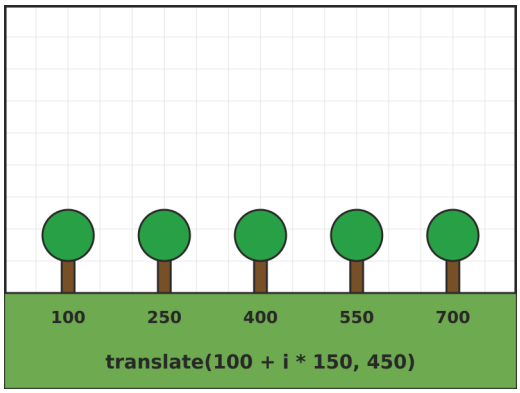
`pop()` also puts back the **fill, stroke and strokeWeight** you had before `push()`.



4 A row of trees

Same row math as Part 1: **`start + i * gap`**. Now the x goes into **`translate()`**.

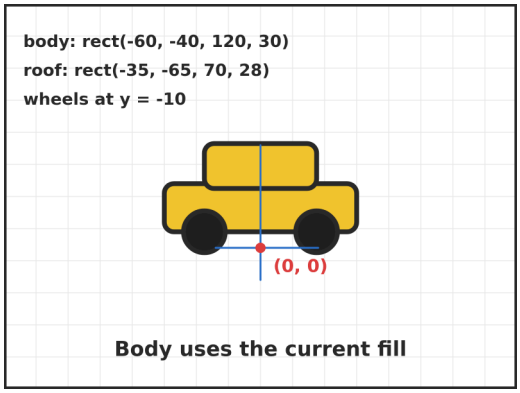
```
for (let i = 0; i < 5; i++) {
  push();
  translate(100 + i * 150, 450);
  drawTree();
  pop();
}
```



5 A shape that takes your color

Leave the **body** fill out of the function. Whatever fill is set before you call it becomes the body color.

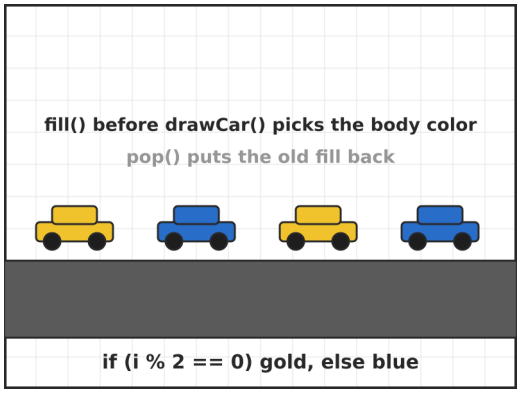
```
function drawCar() {
  rect(-60, -40, 120, 30, 6);    // body
  rect(-35, -65, 70, 28, 6);     // roof
  fill(30);
  circle(-35, -10, 26);          // wheels
  circle(35, -10, 26);
}
```



6 A row of cars

Pick the color inside the loop, then call the function.

```
for (let i = 0; i < 4; i++) {
  push();
  translate(110 + i * 190, 380);
  if (i % 2 == 0) {
    fill(244, 196, 48);
  } else {
    fill(40, 110, 200);
  }
  drawCar();
  pop();
}
```

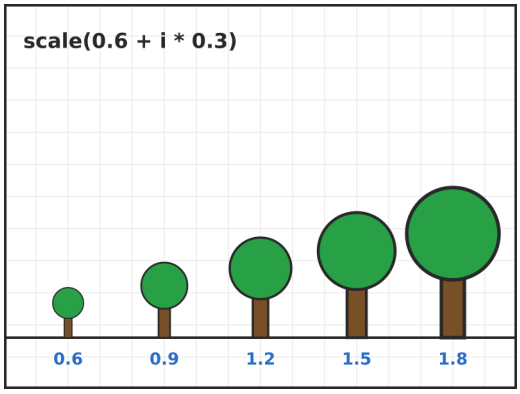


7 scale(s): bigger and smaller

`scale(2)` doubles everything drawn after it. **`scale(0.5)`** halves it. Put it **after** translate.

```
for (let i = 0; i < 5; i++) {
  push();
  translate(100 + i * 150, 520);
  scale(0.6 + i * 0.3);
  drawTree();
  pop();
}
```

Outlines scale too: a big tree gets a thick stroke.

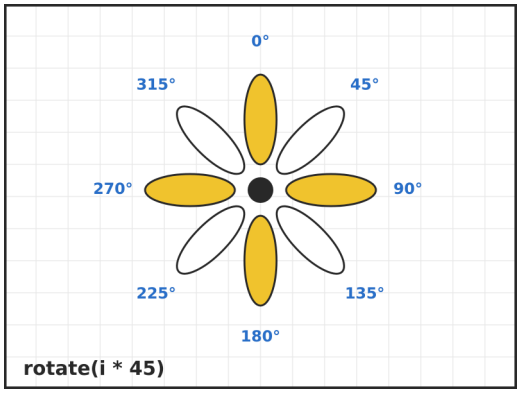


8 rotate(angle): turn around (0, 0)

`rotate()` turns around the **origin**. Translate to the center first, then rotate. Positive angles turn clockwise.

```
angleMode(DEGREES);   // in setup()

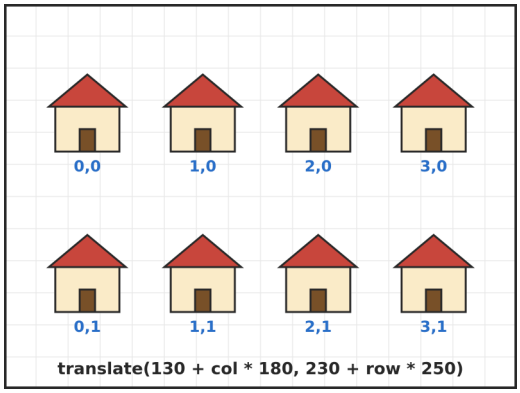
translate(400, 290);
for (let i = 0; i < 8; i++) {
  push();
  rotate(i * 45);
  ellipse(0, -110, 50, 140);
  pop();
}
```



9 A grid of houses

Nested loops from Part 1, with **`drawHouse()`** drawn around (0, 0) like the tree.

```
for (let row = 0; row < 2; row++) {
  for (let col = 0; col < 4; col++) {
    push();
    translate(130 + col * 180, 230 + row * 250);
    drawHouse();
    pop();
  }
}
```

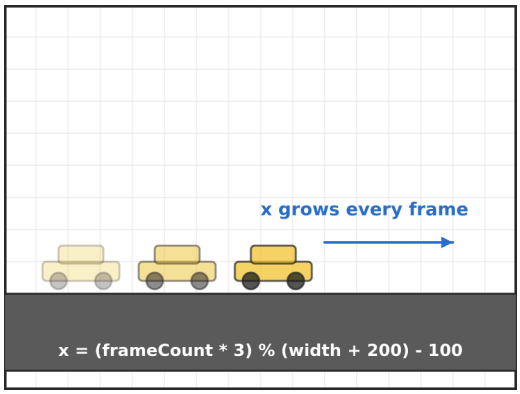


10 Make it move

Put a changing number into **`translate()`**. Here x grows every frame and wraps, so the car drives across forever.

```
let x = (frameCount * 3) % (width + 200) - 100;
push();
translate(x, 440);
fill(244, 196, 48);
drawCar();
pop();
```

The **`+ 200`** and **`- 100`** let it drive in from off screen on the left and off on the right.



Part 2 quick reference

Goal	Code
Move (0, 0)	`translate(x, y)`
Save / restore position and colors	`push(); … pop();`
One copy at each spot in a loop	`push(); translate(…); drawIt(); pop();`
Bigger or smaller	`scale(s)  after translate`
Turn	`angleMode(DEGREES); rotate(degrees)`
Choose the color of a shape	`fill(…) before drawIt()`
Moving copy	`translate(x + frameCount * speed, y)`

Common mistakes

- Forgetting **`pop()`**: every copy lands further away and the colors leak into the next shape. Every **`push()`** needs one **`pop()`**.
- Drawing the shape at its final spot (**`rect(500, 400, …)`**) instead of around (0, 0). Then translate moves it twice.
- Rotating before translating: the shape swings around the top-left corner of the canvas.
- Using radians by accident: **`rotate(45)`** without **`angleMode(DEGREES)`** turns 45 radians.
- Setting the body fill inside the function: then every copy is the same color.

drawHouse() from section 9

```
function drawHouse() {
  fill(250, 235, 200);
  rect(-50, -70, 100, 70);                // walls
  fill(200, 70, 60);
  triangle(-60, -70, 60, -70, 0, -120);   // roof
  fill(120, 80, 40);
  rect(-12, -35, 24, 35);                 // door
}
```